Applied Intelligence
https://doi.org/10.1007/s10489-022-03385-8

UGMINE: utility-based graph mining

Md. Tanvir Alam[1] · Amit Roy[1] · Chowdhury Farhan Ahmed[1] · Md. Ashraful Islam[1] · Carson K. Leung[2]

Accepted: 13 February 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract
Frequent pattern mining extracts most frequent patterns from databases. These frequency-based frameworks have limitations in representing users' interest in many cases. In business decision-making, not all patterns are of the same importance. To solve this problem, utility has been incorporated in transactional and sequential databases. A graph is a relatively complex but highly useful data structure. Although frequency-based graph mining has many real-life applications, it has limitations similar to other frequency-based frameworks. To the best of our knowledge, there is no complete framework developed for mining utility-based patterns from graphs. In this work, we propose a complete framework for utility-based graph pattern mining. A complete algorithm named UGMINE is presented for high utility subgraph mining. We introduce a pruning technique named RMU pruning for effective pruning of the candidate pattern search space that grows exponentially. We conduct experiments on various datasets to analyze the performance of the algorithm. Our experimental results show the effectiveness of UGMINE to extract high utility subgraph patterns.

Keywords Pattern mining · Graph mining · High utility pattern mining

1 Introduction

Discovering frequent patterns from a given database is a well-studied problem in data mining. Frequent pattern mining methods have been developed for transactional databases [11], sequential databases [24] and graph databases [15, 42]. A transactional database is a collection of unordered itemsets. Frequent itemset mining algorithms discover itemsets that frequently appear together in transactions. Algorithms designed for sequential databases discover subsequences that appear frequently from a collection of sequences. Similar to sequential databases, frequent subgraphs are discovered from graph databases.

Frequent pattern mining methods have a major drawback: they always assume the frequency is the only parameter of interest to the user. However, there are many real-life applications in which this assumption does not hold. Let us consider a transactional database of a shop. Each transaction in the database captures merchandise items bought by a customer. An itemset {Milk, Bread} may have a higher frequency in the database than an itemset {Computer, Monitor}. However, the latter may be more interesting to the shop owner as it yields more profit than the former. From this scenario, we can observe that frequency alone may not always be able to capture the users' interest. To solve this problem, utility-based pattern mining is needed.

In utility-based pattern mining, a utility function assigns a utility value to a pattern. This utility value represents the relative importance—in terms of frequency, profit, etc.—of the pattern to the user. Utility-based pattern mining can be considered as a generalization of frequent pattern mining. By designing the utility function, a user can define his interest, which may vary depending on the application. Given a utility function, the basic task of high utility pattern mining algorithms is to discover all patterns from a database associated with high utility values. In other words, the

✉ Chowdhury Farhan Ahmed
farhan@du.ac.bd

Md. Tanvir Alam
tanvircsedu15@gmail.com

Amit Roy
amitroycsedu@gmail.com

Md. Ashraful Islam
ashraful@du.ac.bd

Carson K. Leung
kleung@cs.umanitoba.ca; Carson.Leung@UManitoba.ca

[1] University of Dhaka, Dhaka, Bangladesh

[2] University of Manitoba, Winnipeg, MB, Canada

Published online: 12 April 2022

algorithms find patterns having more interest or importance to the user based on the user-specified utility function, which expresses his interest.

Various methods [9, 19, 21] have been proposed for mining high utility itemsets from transactional databases. In these algorithms, a quantitative transactional database is taken as input. Each item is assigned with an internal and external utility representing the quantity and quality of the item, respectively. The utility of an item is the product of its internal and external utility. In our shop example, the internal utility can be defined as the number of items bought, and the external utility can be defined as the revenue generated from an item. High utility itemset mining discovers itemsets that generate more revenue. Algorithms have also been proposed for mining high utility subsequences from sequential databases [2, 10, 38, 39, 44].

Unlike transactional and sequential databases, to the best of our knowledge, there is no complete framework proposed for utility-based subgraph mining. A graph is a highly useful data structure in many real-life applications. A labeled graph can be used to represent a wide variety of data types and relationships between data entities. For example, to represent a chemical compound, we can use a graph with nodes labeled with atom names and edges labeled with bond types. Web access logs and social networks can also be represented using graphs. Hence, graphs can be considered as a generalized version of transactional and sequential databases as these databases can be represented using graphs. Similar to itemset mining, there are cases where frequency alone is incapable of representing users' interest in graphs.

Let us consider a real-life application of utility-based graph mining. In the domain of information retrieval, to preserve complex associations between words, documents are presented as graphs [22]. Here, each term in a document is represented by a node in the corresponding graph, and an edge between two nodes is added if the corresponding terms occur within the same window of a predefined size. Here, the external utility can be assigned to each edge according to the terms' importance (tf-idf, node centrality) in the document, and internal utility is the number of common occurrences of the terms within the same window. Mining high utility subgraphs from such a database will find complex associations between words preserving their importance in the documents, which can be used for tasks like classification, clustering, and so on.

We can consider another real-life application in the context of online web page advertisements. Here, the activities of a user can be represented by a graph, where the nodes represent the web pages. An edge between two nodes represents that one web page references another web page through an advertisement. The nodes and edges can be labelled according to the advertisement type. Each edge is assigned with (a) an internal utility representing the referral counts and (b) an external utility representing the revenue from the advertisement on its type. For this application, high utility subgraphs represent the portions of the advertisement network from which higher revenue is generated.

Mining interesting substructures from chemical structure databases and web page access logs can be a third notable application of utility-based graph mining. As there is no complete framework proposed for high utility subgraph mining, before devising an algorithm, we need to establish a new framework for this problem first. In this work, we propose a generic framework for utility-based graph mining involving internal and external utility. Then, we also develop an algorithm—named UGMINE—for mining high utility subgraphs from a graph database. Note that subgraph isomorphism checking and exponential candidate generation can make any kind of subgraph mining a costly task. To elaborate, subgraph isomorphism testing is an NP-hard problem. However, one needs to perform a subgraph isomorphism test for each candidate subgraph in each graph. Thus, for each false candidate generated in subgraph mining, there will be significant performance overhead. Hence, we need to construct the search space in such a way that not too many false candidates are generated. Recall from frequent subgraph mining that any subgraph of a frequent subgraph will always be frequent. This property, called the downward closure property, helps to prune false candidates and improve the algorithmic performance significantly. However, the downward closure property does not hold for utility-based pattern mining. To address this problem, we develop search space pruning techniques based on whether extensions are possible or not for a candidate subgraph. Moreover, we also conduct experiments on graph databases to analyze the efficiency of our algorithm on different pruning techniques.

Our *key contributions* in this work can be summarized as:

- A complete framework suitable for utility-based subgraph mining, which incorporates both internal and external utility as defined in high utility pattern mining literature.
- A complete algorithm—called UGMINE—for mining high utility subgraphs from transactional labeled graph databases.
- A tighter pruning technique—named RMU-Pruning—to eliminate false-candidates from the search space and improve runtime performance of the algorithm when compared to the state-of-the-art approaches.

We describe the other recent works—which are closely related to weighted and utility-based graph mining—in Section 2. In Section 3, we propose the framework and the UGMINE algorithm for utility-based graph mining. We present our experimental results and analysis in Section 4.

Finally, we present our conclusion and future work in Section 5.

2 Related works

In this section, we discuss the state-of-the-art solutions and analyze their limitations. We cover various existing frequent pattern mining, weighted frequent pattern mining, utility based pattern mining, graph mining methods related to our work.

2.1 Frequent pattern mining

Mining frequent patterns from a transaction database where a user-defined support threshold is given is a well-studied problem in data mining. An itemset is called frequent if its occurrence frequency satisfies the user-defined minimum support threshold. Let us consider an example transaction database D in Table 1. The support of an itemset X in a transaction T_j is defined as:

$$sup(X, T_j) = \begin{cases} 1 & \text{if } X \subseteq T_j \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

The support of an itemset X in a transaction database D is defined as:

$$sup(X, D) = \sum_{T_j \in D} sup(X, T_j). \quad (2)$$

In our example transaction database D in Table 1, itemset {b,c} and itemset {d} has the support count 2 and 1, respectively. Given a support threshold δ, frequent itemset mining algorithms find all itemsets I in a database D such that

$$sup(I, D) \geq minsup. \quad (3)$$

Here, $minsup = |D| \times \delta$. If the user given support threshold is 66%, then, {b,c} is a frequent itemset and {d} is not a frequent itemset.

Two famous approaches exist for frequent pattern mining, namely the candidate-generation-and-test-based Apriori approach and the tree-based pattern growth approach. Apriori [32] is a frequent pattern mining algorithm which is based on the downward closure property. This property suggests that if an itemset is frequent – that is, its support count satisfies the user-defined minimum support – then all of its subsets will also be frequent. Contrapositively, if an itemset is not frequent, then all of its supersets cannot be frequent. This property is also called the anti-monotonicity property because if a set cannot pass a test then neither can any of its supersets. Apriori based approaches generate candidates while mining frequent itemsets and the downward closure property is used for pruning false candidates which results in performance improvement. The major limitation of Apriori based candidate-generation-and-test algorithms is that they generate too many false candidates when the support threshold is very low. This results in a more time-costly database scan to find the support counts of candidates. To overcome this limitation, a tree-based pattern growth approach, namely FP-growth [11], was proposed. FP-growth first sorts the items in each transaction in descending order of frequency. A prefix tree called an FP-tree is built with the items of the transaction. Then, the FP-tree is mined recursively to find frequent itemsets. Each item in the FP-tree which satisfies the minimum support threshold is declared as a frequent itemset. After that, for each item, a conditional pattern base is created with those items in the branches of the FP-tree which have the item as a suffix. With the conditional pattern base, the mining process continues recursively until the tree contains only a single path. All the combinations of the items in the single path are a frequent itemset. FP-growth does not generate any false candidates and only needs to scan the database twice; in other words, the number of database scans required is constant. However, FP-growth does not work well for incremental databases and is not always memory efficient. Frequent pattern mining methods have also been developed for sequential databases; GSP [31] and PrefixSpan [24] are two such proposed algorithms.

Table 1 A transaction database

Transaction Id	Transaction
T_0	a, b, c
T_1	b, c
T_2	b, d

2.2 Weighted frequent pattern mining

To incorporate the relative importance of items or data entities, the concept of weight has been introduced to frequent pattern mining before utility. In traditional frequent pattern mining, the importance of all patterns is equal, which is not suitable for many real-life applications. Weighted frequent pattern mining finds patterns with higher relative importance to the user, where the relative importance of items are assigned as a weight values according to the application. In Table 2, we show a weighted transaction database. Compared to the transaction database used for traditional frequent pattern mining, here a weight function w assigns a weight value to each item. For example, the item a is assigned with weight value 5. This weight value is used to encode the relative importance of the item. For example, in this database, a is 5 times more important than item d.

Table 2 A weighted transaction database

Id	Transaction
T_0	a, c, d, e
T_1	b, c, e
T_2	d

Item	Weight
a	5
b	11
c	4
d	1
e	2

The weighted support of an itemset X in a weighted transaction database D is defined as:

$$wsup(X, D) = \sum_{i \in X} w(i) \times sup(X, D). \quad (4)$$

The weighted support of the itemset $\{c, e\}$ in D, $wsup(\{c, e\}, D) = (4 + 2) \times 2 = 12$. Similarly, $wsup(\{b, e\}, D) = (11 + 2) \times 1 = 13$. Despite having a lower support count than $\{c, e\}$, the itemset $\{b, e\}$ has a higher weighted support value. Because the itemset $\{b, e\}$ has higher relative importance to user. Similar to frequent pattern mining, weighted frequent pattern mining discovers patterns with weighted support higher than a given threshold.

Weighted frequent pattern mining is useful to discover a smaller number of patterns but with higher significance. Several works have been proposed for mining weighted frequent itemsets. Examples include an Apriori based approach for mining weighted frequent itemsets and association rules [5], a projection based method [17], and a tree based method [36]. Moreover, efficient approaches for mining weighted frequent itemsets and association rules [33, 40], as well as weighted frequent pattern mining algorithms for sequential databases [16, 45, 46], have been proposed.

2.3 Utility-based pattern mining

Although weighted frequent pattern mining addresses the problem of preserving relative importance, it has a limitation that the importance of any pattern over different transactions is the same. For example, in the database of Table 2, the weight of the itemset $\{c, e\}$ is 6 in both the transactions T_0 and T_1. To overcome this limitation, high utility pattern mining is applied to many applications. It assigns utility values to patterns that reflect the interest of users. Profits of shop items and time spent on webpages are two examples of such utility. High utility itemset mining algorithms take a quantitative transaction database (Table 7) and a minimum utility threshold as an input. Each item i is assigned with a positive number $p(i) \in R_+$ called its external utility in a quantitative transaction. Each item i in transaction T_j is also associated with an internal utility, a positive number $q(i, T_j) \in R_+$.

Given a quantitative transaction database, the utility of an item i in a transaction T_j is defined as:

$$u(i, T_j) = q(i, T_j) \times p(i). \quad (5)$$

The utility of an itemset X in a transaction T_j is defined as:

$$u(X, T_j) = \begin{cases} \sum_{i \in X} u(i, T_j) & \text{if } X \subseteq T_j \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

The utility of an itemset X in a quantitative transaction database D is defined as:

$$u(X, D) = \sum_{T_j \in D} u(X, T_j). \quad (7)$$

High utility itemset mining algorithms find all itemsets X in a database D with

$$u(X, D) \geq minutil, \quad (8)$$

where the minimum utility value *minutil* is computed by:

$$minutil = \delta \times u(D). \quad (9)$$

Here, δ is a utility threshold defined by the user and $u(D)$ is defined as:

$$u(D) = \sum_{T_q \in D} tu(T_q). \quad (10)$$

Let us consider a itemset $I = \{c, e\}$ from the quantitative transaction database of Table 3. The utility of item c in T_0 is $u(c, T_0) = 3 \times 4 = 12$. Similarly, $u(c, T_1) = 2 \times 4 = 8$, $u(e, T_0) = 2 \times 2 = 4$, $u(e, T_1) = 5 \times 2 = 10$. The utility of itemset I in T_0 is $u(I, T_0) = u(c, T_0) + u(e, T_0) = 12 + 4 = 16$. Similarly, $u(I, T_1) = 18$ and $u(I, T_2) = 0$. So, utility of I in the database is $u(I) = u(I, T_0) + u(I, T_1) + u(I, T_2) = 16 + 18 + 0 = 34$.

An algorithm named two-phase [21] was proposed which takes a quantitative transaction database (Table 3) as input. The task of the algorithm is to discover all high utility itemsets as defined by (8). Pruning the search space is comparatively difficult in high utility itemset mining versus

Table 3 A quantitative transaction database

Transaction Id	Transaction
T_0	(a, 1), (c, 3), (d, 4), (e, 2)
T_1	(b, 3), (c, 2), (e, 5)
T_2	(d, 2)

Item	External utility
a	5
b	3
c	4
d	1
e	2

frequent itemset mining. In frequent itemset mining, if itemset X is a subset of itemset Y then if X is infrequent, Y will also be infrequent. Hence, we do not need to test any superset of an itemset if it is not frequent. This property of being monotone helps to prune the search space. Unfortunately, we cannot prune the search space in high utility itemset mining using this property. For example, in the quantitative transaction database of Table 3, $u(\{e\}) = 14$ whereas $u(\{c, e\}) = 34$. But $\{e\} \subset \{c, e\}$. So, search space pruning for efficient mining is difficult in this case. The algorithm proposed in [21] uses a measure for itemsets named transaction weighted utilization (TWU) to prune the search space. The transaction utility of a transaction T_j in TWU is defined as:

$$TU(T_j) = \sum_{i \in T_j} u(i, T_j). \tag{11}$$

The transaction weighted utilization of an itemset X is defined as:

$$TWU(X) = \sum_{X \subseteq T_j} TU(T_j). \tag{12}$$

For the quantitative transaction database of Table 3, TU(T_0) = 25, TU(T_1) = 27 and TU(T_2) = 2. So, TWU({c, e}) = TU(T_0) + TU(T_1) = 52. The TWU measure for any itemset X is an upper bound on the utility value of the itemset. That is TWU(X) $\geq$ u(X), for any itemset X. Another interesting property of TWU measure is that TWU(X) $\geq$ TWU(Y) if X $\subseteq$ Y. Moreover if TWU(X) $< minutil$, then any itemset Y $\supseteq$ X is not a high utility itemset. This monotone property helps to reduce the search space effectively in the two-phase algorithm.

In phase one, the algorithm generates all candidate high utility itemsets. An itemset X is a candidate high utility itemset if TWU(X) $\geq minutil \times u(D)$ (by (8)). Using the monotone property of TWU measure reduces the search space here, which is similar to the pruning technique used in the Apriori algorithm. In phase two, the utility of all candidate high utility itemsets is measured using (7). As rejected candidate high utility itemsets cannot be high utility itemsets, this final scan always produces a complete result. The filtering done in this phase reduces runtime and memory required.

Various other works also address the problem of high utility itemset mining [8, 19, 23, 25, 28–30, 35, 41]. HUC-Prune [3] is an algorithm proposed for finding high utility itemsets from databases which follows a tree-based pattern growth approach. Ahmed et al. [1] proposed methods for mining high utility itemsets from incremental databases. Algorithms for high average-utility itemsets have been proposed in [34, 47]. Algorithms have also been proposed for mining high utility subsequences from sequential databases [2, 20, 44, 48, 49].

2.4 Graph mining

To mine frequent subgraphs from transactional graph databases, the gSpan [42] algorithm is the most famous approach. It has been used in temporal subgraph mining [4, 27], uncertain subgraph mining [6], correlated subgraph mining [7], weighted subgraph mining [12], along with utility-based subgraph mining in a distributed platform [14], and others.

In general, the gSpan algorithm starts with an empty subgraph and gradually extends it by adding edges. It follows the rightmost path extension for edge extension. However, there could be duplicate subgraphs in the rightmost path extension of a candidate graph. The gSpan algorithm uses a depth-first search (DFS) code for each candidate and for the isomorphic candidates. It eliminates every duplicate candidate, except the one whose DFS code is canonical or minimal.

In a tree approach following the DFS code, each vertex u is assigned a DFS order index d_u according to the DFS discovery time. Each edge (u, v) is represented as a DFS code tuple $(d_u, d_v, l(u), l(v), l(u, v))$. An edge (u, v) is a forward edge if $d_u < d_v$. Otherwise, it is a backward edge. Let $t_1 = (d_{u1}, d_{v1}, l(u_1), l(v_1), l(u_1, v_1))$ and $t_2 = (d_{u2}, d_{v2}, l(u_2), l(v_2), l(u_2, v_2))$ be two DFS code tuples. Then, $t_1 < t_2$ if and only if one of the following holds:

i. $(d_{u1}, d_{v1}) = (d_{u2}, d_{v2})$ and $(l(u_1), l(v_1), l(u_1, v_1)) <_e (l(u_2), l(v_2), l(u_2, v_2))$, where e is a lexicographic order on labels.
ii. $d_{u1} < d_{v1}$ and $d_{u2} < d_{v2}$ and $d_{v1} < d_{v2}$
iii. $d_{u1} < d_{v1}$ and $d_{u2} < d_{v2}$ and $d_{v1} = d_{v2}$ and $d_{u1} > d_{u2}$
iv. $d_{u1} > d_{v1}$ and $d_{u2} > d_{v2}$ and $d_{u1} < d_{u2}$
v. $d_{u1} > d_{v1}$ and $d_{u2} > d_{v2}$ and $d_{u1} = d_{u2}$ and $d_{v1} < d_{v2}$
vi. $d_{u1} < d_{v1}$ and $d_{u2} > d_{v2}$ and $d_{v1} \leq d_{u2}$
vii. $d_{u1} > d_{v1}$ and $d_{u2} < d_{v2}$ and $d_{u1} < d_{v2}$

The DFS code of a graph consists of ordered DFS code tuples associated with edges. A partial order between DFS codes can be defined by comparing tuple-by-tuple. For a labeled graph G, we define min(G) as the minimum DFS code of G according to a defined order. This is also called the canonical code for a subgraph.

In a DFS code for a graph, the vertex with the highest DFS order index is called the rightmost vertex. The path from the root to rightmost vertex containing only forward edges is called the rightmost path. The DFS code tree uses an edge-growth approach to extend the DFS code of a candidate subgraph starting from an empty graph. In the DFS code tree, a backward edge extension is done only from the rightmost vertex to one of the vertices in the rightmost path. A forward edge extension is done only from the vertices in the rightmost path to a vertex that has not already existed in the candidate subgraph. Non-canonical codes are not extended further. In this way, the DFS code

tree covers canonical codes of all the candidate subgraphs. We also use the rightmost path extension to generate the candidates in our algorithm.

As research in graph mining progresses, the researchers tend to incorporate weight or utility into the edges and nodes of graphs. Since the gSpan [42] algorithm does not consider the weight property of edges, the algorithm is not practical for mining weighted frequent subgraphs from weighted graph databases. Different researchers incorporated the weight property into the graph in different manners. In [13], different weighting techniques are proposed—namely ATW, AW and UBW, where ATW does not differentiate between two subgraphs if they have same support. Both AW and UBW use affinity-based and utility-based weight functions, respectively. Recently, several researches have been conducted in multi-weighted [26] subgraph mining that uses different weighting functions to assign weight in nodes and edges with both exact and approximate solutions. In contrast, our proposed approach uses a utility function for each edge involving both the node label and edge label.

Moreover, WIGM [43] proposed an approach to mine frequent weighted graph from single weighted graphs. In contrast, our approach mines high utility subgraphs from a set of labeled graph databases.

To prune the candidates, the downward closure property is used in different pattern mining algorithms. However, this property is not held by the weighted support used in weighted frequent subgraph mining, which leads to a challenge. To address this challenge, Weighted Frequent Subgraph Mining with the use of the Max-Possible Weighted Support condition for pruning (WFSM-MaxPWS) [12] was proposed. In contrast, our approach has a different framework and incorporates the concept of utility in terms of node label and edge label. Weighted subgraph mining algorithm has been proposed for single large graph also [18].

By estimating an upper bound for pruning each generated candidates, a distributed approach [14] was proposed for utility-based subgraph mining. However, it does not consider the internal and external utility separately, and it ignores the relationship between vertex or edge label and the profit or utility value. In contrast, our approach uses a more effective pruning technique, namely RMU-prune. Our framework is suitable to handle internal and external utility.

3 Proposed methods

As a preview, we will define a framework for utility-based subgraph mining in Section 3.1, and present a complete algorithm for mining utility-based subgraphs in Section 3.2.

3.1 Proposed utility-based graph mining framework

Let us define the utility-based subgraph mining framework formally. Let D be a quantitative labeled graph database containing a set of quantitative labeled graphs, a set of labels L and a function $q : L \times L \times L \rightarrow R_+$. A quantitative labeled graph $G \in D$ can be represented by 4-tuple (V, E, l, p), where (a) V is a set of vertices; (b) $E \subseteq V \times V$ is a set of edges; (c) $l : V \cup E \rightarrow L$ is a function that labels vertices and edges; and (d) $p : E \rightarrow R_+$ is a function that gives the internal utility of an edge.

Let $e \in E$ be an edge with vertices $u, v \in V$. The internal utility $p(e)$ of e represents quantity. The external utility $q(l(u), l(v), l(e))$ of e represents the quality of the edge. The utility $u_e(e)$ of an edge e with vertices u, v is then defined as:

$$u_e(e) = p(e) \times q(l(u), l(v), l(e)). \tag{13}$$

The utility $u_g(g)$ of a quantitative labeled subgraph $g = (V_g, E_g, l, p)$ is defined as:

$$u_g(g) = \sum_{e \subset E_g} u_e(e). \tag{14}$$

A subgraph isomorphism from a labeled subgraph $g' = (V'_g, E'_g, l')$ to a quantitative labeled subgraph $g = (V_g, E_g, l, p)$ holds if there exists a bijective function $\phi : V'_g \rightarrow V_g$ such that

1. $(u, v) \in E'_g \iff (\phi(u), \phi(v)) \in E_g$;
2. $\forall u \in V'_g, l(u) = l(\phi(u))$; and
3. $\forall (u, v) \in E'_g, l(u, v) = l(\phi(u), \phi(v))$.

Let $\phi(g', G)$ be a function that returns all the quantitative labeled subgraphs g such that a subgraph isomorphism from g' to g holds. The utility $u_G(g', G)$ of a labeled subgraph g' in a quantitative labeled graph G is defined as:

$$u_G(g', G) = \max_{g \in \phi(g', G)} u_g(g). \tag{15}$$

The utility $u_D(g', D)$ of a labeled subgraph g' in a quantitative labeled graph database D is defined as:

$$u_D(g', D) = \sum_{G \in D} u_G(g', G). \tag{16}$$

Given a quantitative labeled graph database and a threshold δ defined by the user, the task of high utility subgraph mining is to discover all labeled subgraphs g' such that $u_D(g', D) \geq minutil$ where $minutil$ is defined as:

$$minutil = \sum_{G \in D} u_g(G) \times \delta. \tag{17}$$

In Table 4, we presented the key definitions of the framework. A quantitative labeled graph database D containing

Table 4 Definitions

Term	Definition
$u_e(e)$	The utility of an edge e
$u_g(g)$	The utility of a quantitative labeled subgraph g
$u_G(g', G)$	The utility of a labeled subgraph g' in a quantitative labeled graph G
$u_D(g', D)$	The utility of a labeled subgraph g' in a quantitative labeled graph Database D
ϕ(g', G)	Subgraph isomorphisms from labeled subgraph g' to quantitative labeled graph G
δ	User defined utility threshold

two quantitative labeled graphs G_1 and G_2 is presented in Fig. 1. Each vertex in the graphs is assigned with a label and an identification number. Each edge is assigned with a label and a number representing internal utility.

In Fig. 1, the internal utility of edge (0,1) in graph G_1 is 3. The edge label is q, and the vertices labels are a and b. From Table 5, we find that external utility of such an edge is 3. So, the utility of edge (0, 1) in graph G_1 is 3×3 = 9. The utility of graph G_1 is $u_e(0, 1) + u_e(0, 2) + u_e(1, 2)$ = 3×3 + 4×4 + 5×2 = 35. Similarly, the utility of graph G_2 is 16. So, The utility of the database D is $u_g(G_1) + u_g(G_2)$ = 35 + 16 = 51. If δ is 0.35, then $minutil$ = 51 × 0.35 = 17.85. Let the labeled subgraph presented in Fig. 2 be g'. A subgraph isomorphism ϕ_1 from g' to a subgraph $g \in G_1$ holds where $\phi_1(0) = 2$, $\phi_1(1) = 0$ and $\phi_1(0) = 2$. So, $u_G(g', G_1) = u_g(g) = u_e(0, 1) + u_e(0, 2) = 3\times3 + 4\times4 = 25$. Similarly, we find that $u_G(g', G_2) = 14$ and $u_D(g', D) = u_G(g', G_1) + u_G(g', G_2) = 25 + 14 = 39$. As $u_D(g', D)$ is greater than $minutil$, g' is a high utility subgraph in Database D.

3.2 Proposed algorithm

Here, we propose a naive algorithm UGMINE for extracting high utility subgraph patterns from graph databases as defined in the framework in Section 3.1. The major challenges of high utility subgraph mining are candidate labeled subgraph generation and efficient pruning of search space. For candidate generation, we consider an edge growth approach where an edge is added each time starting from an empty graph to build the search tree. The problem with this approach is that many duplicate isomorphic subgraph candidates are generated. To solve this problem, we use the DFS code tree approach used in the gSpan algorithm. To prune the search space more effectively, we define a graph weighted utility (GWU) value for all candidate subgraphs.

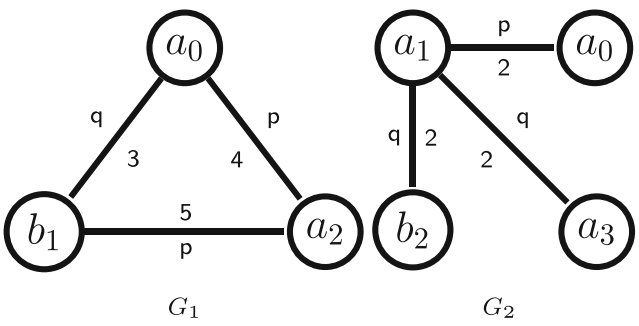


Fig. 1 Sample Database D

Definition 1 The graph weighted utility (GWU) value $\mathrm{GWU}(g', G)$ of a labeled subgraph g' in a quantitative labeled graph G is defined as:

$$GWU(g', G) = \begin{cases} u_g(G) & \text{if } \exists g \in \phi(g', G) \\ 0 & \text{otherwise.} \end{cases} \tag{18}$$

Definition 2 The GWU value of a labeled subgraph g' in a quantitative labeled graph database D is defined as:

$$GWU(g', D) = \sum_{G \in D} GWU(g', G). \tag{19}$$

For example, the GWU value of the labeled subgraph g' from Fig. 2 in the database D of Fig. 1 is GWU(g', D) = 35 + 16 = 51.

For all canonical codes in DFS code tree, UGMINE calculates the utility of the labeled subgraph associated with the code as defined in (16). If the utility value is greater than or equal to $minutil$, then the labeled subgraph is a high utility subgraph. In UGMINE, we prune codes in the DFS code tree that are associated with non-high GWU subgraphs.

3.3 RMU Pruning

In UGMINE, we use GWU (19) value based pruning. Now, we propose a more effective pruning technique by establishing a tighter upper bound on the utility of a candidate

Table 5 External utility

$l(u), l(v), l(e)$	Ext. Utility
a,a,p	4
a,a,q	1
a,b,p	2
a,b,q	3



Fig. 2 g'

subgraph and all of the supergraphs extended from it. We observe that an edge adjacent to a vertex—which is not on the rightmost path of a candidate subgraph—will not be added in any of its extensions.

Definition 3 Let $g' = (V'_g, E'_g, l'_g)$ be a candidate labeled subgraph in a DFS code tree with the rightmost path R'_g. Let G be a quantitative labeled graph and ϕ be an isomorphism from g' to $g \in G$. We define, $LM(g', G, g) = \{(u, v) : (u, v) \in E_G, \phi^{-1}(u, v) \notin E'_g, \phi^{-1}(u) \in V'_g - R'_g$ or $\phi^{-1}(v) \in V'_g - R'_g\}$.

We define the rightmost utility (RMU) value for all candidate subgraphs.

Definition 4 The RMU value $RMU(g', G)$ of a labeled subgraph g' in a quantitative labeled graph G is defined as:

$$RMU(g', G) = \max_{g \in \phi(g', G)} \{u_g(G) - \sum_{e \in LM(g', G, g)} u_e(e)\}. \quad (20)$$

Definition 5 The RMU value $RMU(g', D)$ of a labeled subgraph g' in a quantitative labeled graph database D is defined as:

$$RMU(g', D) = \sum_{G \in D} RMU(g', G). \quad (21)$$

Definition 6 Given a quantitative labeled graph database and a minimum utility value $minutil$, a labeled subgraph g' is a high RMU subgraph if and only if RMU(g', D) $\geq minutil$.

Theorem 1 *The high RMU subgraph is antimonotonic.*

Proof Given any two labeled graphs g'$_1$, g'$_2$ in a DFS tree and any quantitative labeled graph database D such that g'$_1$ is a subgraph of g'$_2$, we need to prove that, if RMU(g'$_1$,D) $< minutil$, then RMU(g'$_2$,D) $< minutil$. For a quantitative labeled graph G∈D, if $\nexists g_1 \in \phi(g'_1, G)$, then $\nexists g_2 \in \phi(g'_2, G)$ and RMU(g'$_1$,G) = RMU(g'$_2$,G) = 0. If $\exists g_1 \in \phi(g'_1, G)$ and $\nexists g_2 \in \phi(g'_2, G)$, then RMU(g'$_1$,G) $\geq$ RMU(g'$_2$,G) = 0. Then, we examine the case where $\exists g_1 \in \phi(g'_1, G)$ and $\exists g_2 \in \phi(g'_2, G)$. Let $\phi_2(g'_2) = g_2$ be the isomorphism in $\phi(g'_2, G)$ where $\sum_{e \in LM(g'_2, G, g_2)} u_e(e)$ is minimum. Note that $\phi_2(g'_1) \in \phi(g'_1, G)$ and $\phi_2(g'_1)$ is a subgraph of g_2. Let e be an edge in $LM(g'_1, G, \phi_2(g'_1))$. Then, e$\notin g_2$ because g'_2 is an extension of g'_1 in the DFS tree. In the DFS code tree, a backward edge extension is done only from the rightmost vertex to one of the vertices in the rightmost path. However, e contains at least one vertex that is not on the rightmost path. So, e will not be extended as a backward edge extension. A forward edge extension is done only from the vertices in the rightmost path to a vertex that does not already exist in the candidate subgraph. As e contains at least one vertex that is not on the rightmost path but exists in the candidate subgraph, e will not be extended as a forward edge extension either. Thus, $\phi^{-1}(e) \notin E'_{g_2}$ as e$\notin g_2$. By definition, e$\in G$. As (a) one of the vertices in e is not on the rightmost path of g'_1 and (b) g'_2 is an extension of g'_1 in the DFS tree, one of the vertices in e is not on the rightmost path of g'_2. So, e$\in LM(g'_2, G, g_2)$ and $LM(g'_2, G, g_2)$ contains all the edges in $LM(g'_1, G, \phi_2(g'_1))$. That yields

$$\sum_{e \in LM(g'_2, G, g_2)} u_e(e) \geq \sum_{e \in LM(g'_1, G, \phi_2(g'_1))} u_e(e). \quad (22)$$

According to our assumption, we can write

$$\min_{g \in \phi(g'_2, G)} \sum_{e \in LM(g'_2, G, g)} u_e(e) \geq \min_{g \in \phi(g'_1, G)} \sum_{e \in LM(g'_1, G, g)} u_e(e). \quad (23)$$

which can be rewritten as

$$\begin{matrix} \max_{g_2 \in \phi(g', G)} \left\{ u_g(G) - \sum_{e \in LM(g', G, g_2)} u_e(e) \right\} \leq \\ \max_{g_1 \in \phi(g', G)} \left\{ u_g(G) - \sum_{e \in LM(g', G, g_1)} u_e(e) \right\} \end{matrix} \quad (24)$$

By (20), we obtain

$$RMU(g'_2, G) \leq RMU(g'_1, G).$$

By (21), we can conclude that

$$RMU(g'_2, D) \leq RMU(g'_1, D).$$

So, if RMU(g'$_1$,D) $< minutil$, then RMU(g'$_2$,D) $< minutil$. In other words, if a labeled graph g'$_1$ is not a high RMU subgraph, then any of its supergraphs g'$_2$ cannot be high RMU subgraphs. □

Similar to UGMINE, we prune all non-high RMU subgraphs from the DFS code tree in UGMINE-RMU. In Algorithm 1, we present the UGMINE algorithm with the RMU pruning.

Theorem 2 *The UGMINE-RMU algorithm is complete.*

Proof Similar to UGMINE, we also use the DFS code tree in UGMINE-RMU for candidate generation, which contains minimum DFS codes for all graphs. Next, we need to prove that pruning non-high RMU subgraphs does not prune any high utility subgraphs. For a quantitative labeled graph G∈D, if there exists no subgraph g such that $g \in \phi(g', G)$, then $u_G(g', G)$ = RMU(g',G) = 0.

Let us examine the case where $\exists g \in \phi(g', G)$. By (15), we obtain $u_G(g', G) \leq u_g(G)$. According to the definition, if ϕ is an isomorphism from g' to g∈G, then $LM(g', G, g)$ does not contain any edge in G that is isomorphic to an edge in g'. This leads to

$$u_G(g', G) \leq \max_{g \in \phi(g', G)} \left\{ u_g(G) - \sum_{e \in LM(g', G, g)} u_e(e) \right\}. \quad (25)$$

In other words, in both cases, $u_G(g', G) \leq$ RMU(g',G) yields

$$\sum_{G \in D} u_G(g', G) \leq \sum_{G \in D} RMU(g', G). \quad (26)$$

By (16) and (21), we obtain

$$u_D(g', D) \leq RMU(g', D). \quad (27)$$

Algorithm 1 UGMINE.

```
1:  procedure UGMINE(C, D, minutil)   ▷ Initially C = ∅
2:      E ← RightMostPathExtensions(C, D)
3:      for e ∈ E do
4:          C' ← C ∪ e
5:          if IsCanonical(C') is False then
6:              Continue
7:          g = Graph(C') \\Graph associated with code C'
8:          if u_D(g, D) ≥ minutil then
9:              g is a high utility subgraph
10:         if RMU(g, D) ≥ minutil then ▷ GWU(g, D)
    in case of GWU pruning
11:             UGMINE(C', D, minutil)
```

Algorithm 2 Canonality checking for a DFS code.

```
1:  procedure ISCANONICAL(C)
2:      g' ← Graph(C)
3:      C' ← ∅
4:      for t ∈ C do
5:          m = min(RightMostPathExtensions(C, g'))
6:          if m < t then
7:              return False
8:          C' ← C' ∪ t
9:      return True
```

Algorithm 3 Right most path extensions.

```
1:  procedure RIGHTMOSTPATHEXTENSIONS(C, D)
2:      E = ∅
3:      for G ∈ D do
4:          if C = ∅ then
5:              for (u, v) ∈ G do
6:                  E ← E ∪ (0, 1, l(u), l(v), l(u, v))
7:          else
8:              R ← Vertices on right most path in C
9:              u_r ← Right most vertex in C
10:             g' ← Graph(C)
11:             for φ ∈ SubgraphIsomorphisms(C, G) do
12:                 for v ∈ R do
13:                     if (u_r, v) ∉ g' and (u_r, v) ∈ G then
14:                         E ← E ∪ (u_r, v, l(u_r), l(v), l(u_r, v))
15:                 for u ∈ R do
16:                     if v ∉ g' and (u, v) ∈ G then
17:                         E ← E∪(u, u_r+1, l(u), l(v), l(u, v))
18:     return Set of all distinct tuples in E
```

So, if g' is a non-high RMU subgraph, then $u_D(g', D) \leq RMU(g', D) < minutil$. In other words, a non-high RMU subgraph cannot be a high utility subgraph. By Theorem 1, we can conclude that, if we prune a non-high RMU subgraph g', then all of its supergraphs are also non-high RMU subgraphs and thus non-high utility subgraphs. Hence, the DFS code tree with the RMU pruning contains minimum DFS codes for all high utility graphs. □

In the algorithm UGMINE, we initially take (a) a candidate empty DFS code C, (b) the transactional database D, and (c) a minimum utility threshold *minutil* as the input. In line 2, we find the rightmost path extension E. For each edge e in the rightmost path extension of E, we generate a new DFS code C', adding$e$ to the original candidate C (line 4). If the new candidate is not canonical, the loop has been continued (lines 5-6). Otherwise, we find the candidate subgraph g associated with C' (line 7). If g satisfies the minimum utility threshold *minutil*, it is declared as a high utility subgraph. If the upper bound pruning measure of g—which can be $GWU(g, D)$ or $RMU(g, D)$—satisfies the minimum utility threshold *minutil*, then the procedure UGMINE is called recursively for further extension.

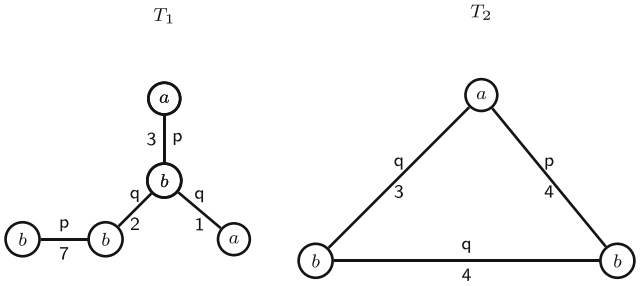


Fig. 3 A quantitative labeled graph database D

Table 6 External utility table

Node label 1	Node label 2	Edge label	External utility
a	b	p	6
a	b	q	2
b	b	p	5
b	b	q	4

3.4 Simulation

Here, we present a complete simulation of our proposed approach. Although some candidates in the search space are not pruned by UGMINE-GWU, they will be pruned by UGMINE-RMU, which supports our claim that UGMINE-RMU has a tighter pruning condition than UGMINE-GWU. Some isomorphic candidates of the search space are also pruned because the same candidates can be generated with a minimal DFS code and we do not want duplicate candidates in the search space.

Figure 3 shows a transactional, labeled graph database with two transactions T_1 and T_2. In each transaction, the nodes and edges are labeled from the set of labels a,b. Each edge in the transactional labeled database has an internal utility associated with it. We also have an external utility table (Table 6) that maps each possible node and edge labels in the transactions to external utility value.

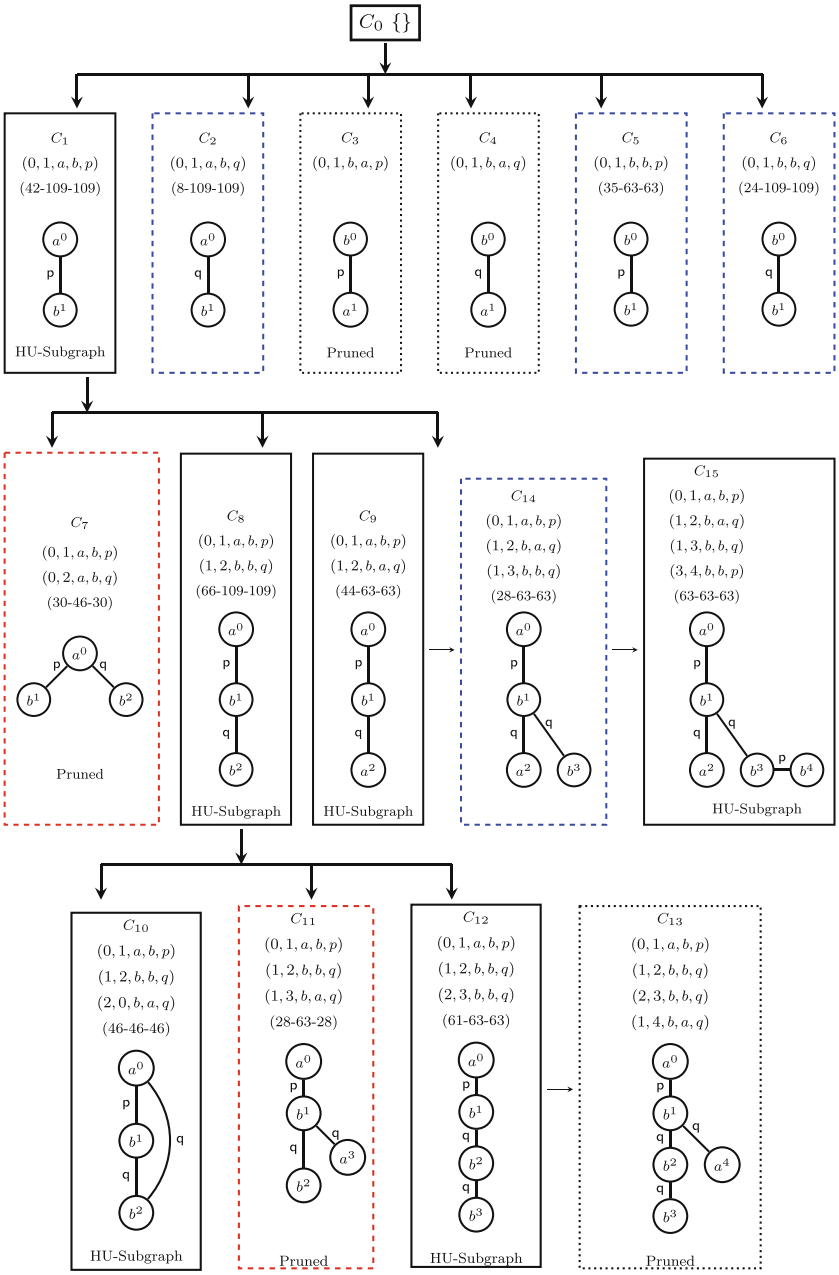


Fig. 4 UGMINE-Simulation for quantitative labeled database D

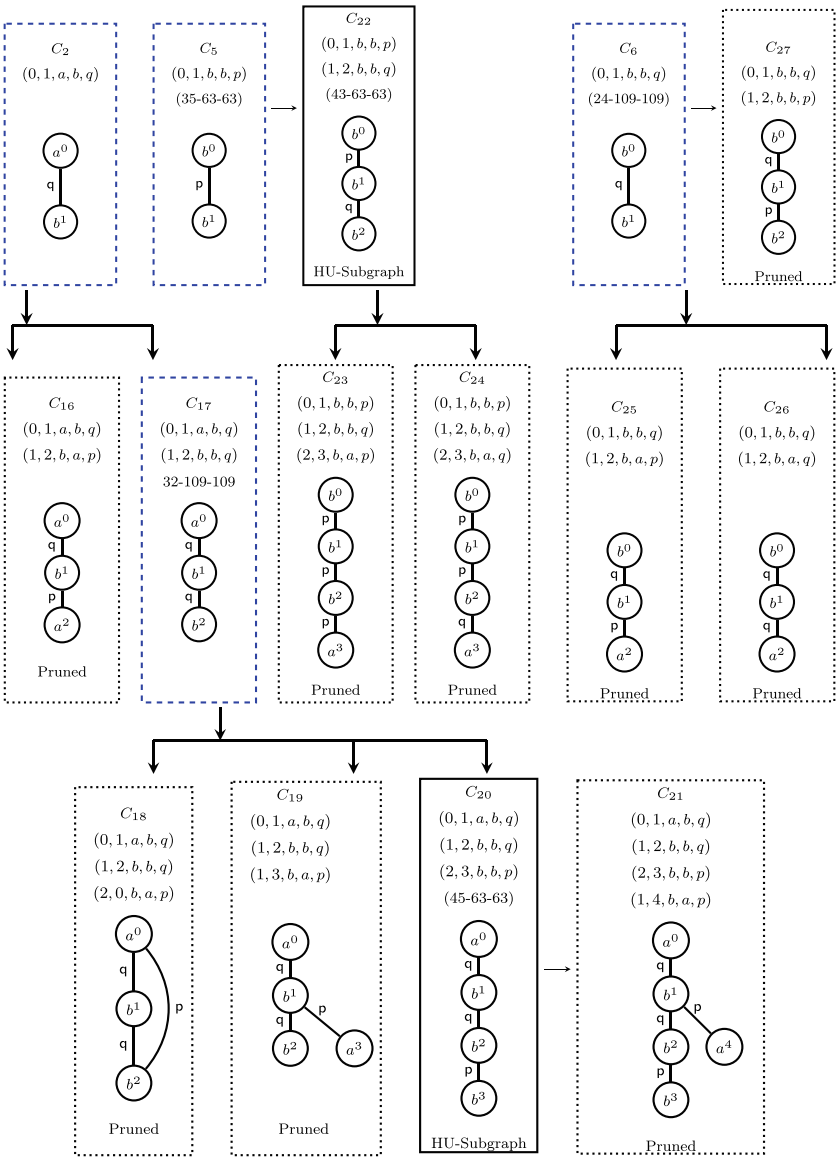


Fig. 5 UGMINE-Simulation for quantitative labeled database D

$U(T_1)$=U((a-b-p)) + U((b-b-q))+U((a-b-q))+U((b-b-p))
= (3 × 6) + (2 × 4) + (1 × 2) + (7 × 5)
= 18 + 8 + 2 + 35 = 63
$U(T_2)$ = U((a-b-p)) + U((b-b-q)) + U((a-b-q))
= (4 × 6) + (4 × 4) + (3 × 2)
= 24 + 16 + 6 = 46
$U(D) = U(T_1) + U(T_2) = 63 + 46 = 109$

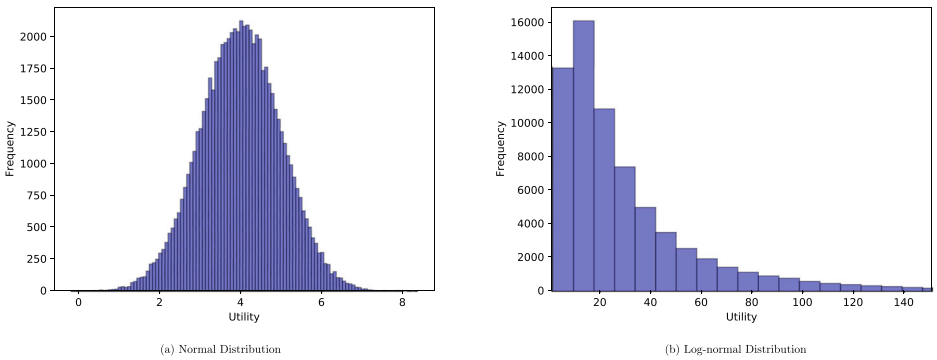
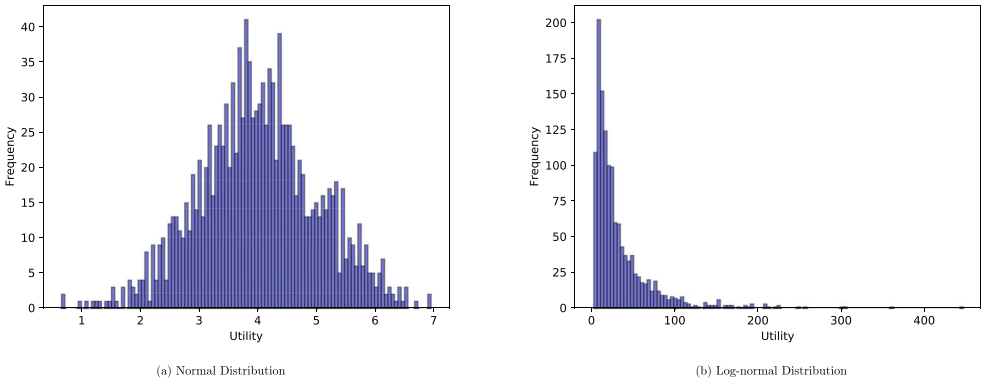


Fig. 6 Internal Utility Weight Distribution: p388



(a) Normal Distribution (b) Log-normal Distribution

Fig. 7 External Utility Weight Distribution: p388

Suppose the minimum utility threshold $\delta = 35\%$. Then, *minutil* = 0.35 × 109 = 38.15. Any subgraph whose utility is greater or equal to *minutil* = 38.15 will be a high utility subgraph. We start from an empty candidate C_0 and generate further candidate subgraphs by extending it with the rightmost path extension approach of gSpan. Each candidate subgraph is represented in a rectangle that has a candidate number, followed by the DFS code of the corresponding candidate subgraph. The next line of each candidate has three real numbers. Those are (subgraph utility-GWU value-RMU value). The graphical representation of each subgraph is formed from the DFS codes. We used solid, dotted and dashed rectangles to represent different types of candidates generated in the search space (Figs. 4 and 5).

The candidate subgraphs with **solid** rectangles are HU-subgraphs as they are not pruned by the GWU or RMU value, and their subgraph utility values also support the minimum utility threshold. They are not pruned, but used for generating candidates by further extension in the search space. Consider the candidate C_1 with the DFS code (0,1,a,b,p). The edge (a-b-p) is present in transactions T_1 and T_2 with internal utility 3 and 4, respectively. From the external utility table, we find that edge (a-b-p) has the external utility 6. So, C_1 has the subgraph utility (4+3) × 6 = 7 × 6 = 42. Again, transactions T_1 and T_2 have transaction utility 63 and 46, respectively, which give the GWU value of C_1 = 63 + 46 = 109. As we cannot exclude any edge adjacent to the rightmost path of C_1, we cannot reduce the upper bound of GWU. So, the RMU value for C_1 remains at 109.

The candidate subgraphs that are isomorphic to other candidate subgraphs are represented by **dotted** rectangles. They are pruned from the search space and not used for generating candidates in further extensions. For example, the candidate subgraph C_3 has the DFS code (0,1,b,a,p) while the candidate subgraph C_1 has the DFS code (0,1,a,b,p). So, C_3 is actually an isomorphism of C_1, and we can safely prune it from the search space as it will not be used for any further extensions.

Candidates with dashed rectangles are those candidates that are not isomorphic to any other candidates, but they are also not high utility subgraphs. We separate the dashed candidates using blue and red colored rectangles.

Candidate subgraphs in the search space with **blue-colored dashed rectangles** are not pruned by the GWU or RMU values, but their subgraph utility is less than the minimum utility value. So, they are not considered high utility subgraphs. As they are not pruned, further extensions from them can generate high utility subgraphs. Consider the candidate subgraph C_2. It has the subgraph utility 8 < *minutil* = 38.15. So, it is not considered as a high utility subgraph. However, it has a GWU and RMU value of 109. Therefore, it is not pruned from the search space but used for further extensions to generate candidates C_{16} and C_{17}. We can show similar analysis for candidate subgraphs with blue-colored dashed rectangles C_5, C_6, C_{14} and C_{17}.

On the other hand, candidate subgraphs with **red-colored dashed rectangles** are those candidates whose GWU value satisfies the minimum utility threshold. They are pruned by the RMU value. This supports our claim that UGMINE-RMU is a tighter pruning method than UGMINE-GWU. Consider the candidate C_7 as an example. It has the DFS code (0,1,a,b,p) (0,2,a,b,q). This subgraph is only present in T_2, which has transaction utility 46. So, the GWU value for C_7 = 46. The rightmost path is now (0-2-a-b-q). The other edge in this candidate has the DFS code (0-1-a-b-p), which is not present in the rightmost path. So, there will be no

Table 7 Statistical description of datasets

Dataset name	Total graphs	Average nodes	Average edges
OVCAR-8	2079	33.69	36.59
Yeast	9568	24.26	25.89
p388	2298	28.01	29.89
SW-620	2410	32.96	35.75
MOLT-4	3140	31.65	34.32
NCI1	4,110	29.87	32.30

Table 8 Runtime and search space reduction statistics: OVCAR

(a) Normal Distribution						
δ	UGMINE-GWU pruning			UGMINE-RMU pruning		
	Runtime (minute)	Candidates	HU Sub-Graphs	Runtime (minute)	Candidates	HU Sub-Graphs
8%	965.60	9139	270	766.16	880	270
8.5%	790.22	6975	224	630.50	660	224
9%	612.18	5428	182	477.59	530	182
9.5%	473.05	4235	151	379.92	374	151
10%	382.45	3282	121	329.02	255	121
(b) Log Normal Distribution						
7%	710.15	608951	873	575.57	475954	873
8%	477.14	389682	611	434.04	317952	611
9%	285.11	277580	435	268.29	231365	435
10%	216.17	209294	335	209.97	177018	335

extension from the vertex b with timestamp 1. According to our claim of the RMU, the utility of any edge in the original transaction that is adjacent to a node not present in the rightmost path can be subtracted from the upper bound of GWU. The utility of edge (b-b-q) is 4×4 = 16 because the edge (b-b-q) is adjacent to the vertex b^1. We can safely subtract the utility of (b-b-q) to calculate the RMU value = 46-16 = 30. The RMU value for C_7 is less than the minimum utility threshold *minutil* of 38.15. Therefore, we can safely prune the candidate C_7 using the RMU-pruning condition. We can also show a similar analysis for the candidate subgraph C_{11}, which is only present in transaction T_1. It has a GWU value of 63, which satisfies the minimum utility threshold minutil = 38.15. The rightmost path in this candidate subgraph is (0,1,a,b,p) (1,3,b,a,q). Another edge present in this subgraph is (1,2,b,b,q). So, the node b^2 in this candidate subgraph is not present in the rightmost path, and any further extension will not include b^2. Therefore, we can safely exclude any edge in the original transaction that is adjacent to b^2 to reduce the upper bound of GWU. b^2 has an adjacent edge (b-b-p), which has the utility $7 \times 5 = 35$. So, the RMU value of C_{11} becomes $63 - 35 = 28$, and we can safely prune C_{11} based on the RMU-pruning condition. Therefore, candidate subgraph C_{11} is not pruned by the GWU-pruning condition, but pruned by the RMU-pruning condition of our framework.

4 Experimental results

For performance evaluation, we have implemented our proposed algorithms in Python and conducted experiments using a PC with an Intel Core i7-6700K CPU at 4.00 GHz and 16 GB RAM. We considered runtime, search space reduction, and memory usage as our performance evaluation metrics. All the datasets, except NCI1, are collected from

Table 9 Runtime and search space reduction statistics: p388

(a) Normal Distribution						
δ	UGMINE-GWU pruning			UGMINE-RMU pruning		
	Runtime (minute)	Candidates	HU Sub-Graphs	Runtime (minute)	Candidates	HU Sub-Graphs
14%	334.99	7606	3	175.59	23	3
14.2%	268.26	6009	2	159.62	23	2
14.4%	242.27	4967	2	152.87	23	2
14.6%	212.12	4190	1	139.41	22	1
(b) Log-normal Distribution						
14%	740.18	682775	5	285.06	271500	5
14.2%	507.86	496761	4	247.97	239266	4
14.4%	344.08	360203	3	226.94	213410	3
14.6%	277.27	311757	3	186.24	194536	3

PubChem[1]. They provide information on the biological activities of small molecules, containing the bioassay records for anti-cancer screen tests with different cancer cell lines. NCI1 [37] is a chemical compound dataset. For utility assignment, we used (a) normal distribution and (b) log-normal distribution (Figs. 6 and 7). The normal distribution is mound-shaped. However, in real life, the distribution of utility to different individuals is right-skewed. For example, there are usually more low-cost products than expensive products in a shop. Hence, we used the log-normal distribution. For both the normal and log-normal distributions, we set μ (mean) = 3.0 and σ (standard deviation) = 1.0. Table 7 shows a statistical description on the total number of graphs, average nodes and edges of the datasets that we used for our experiments.

For OVCAR-8 dataset with normal weight distribution, we ran our proposed algorithms with minimum utility thresholds *minutil* δ = 8%, 8.5%, 9%, 9.5% and 10%. For log-normal distribution of weight, we used minimum utility thresholds *minutil* δ = 7%, 8%, 9% and 10%. The required runtime, number of candidates generated, and number of high utility patterns mined in UGMINE-GWU and UGMINE-RMU for both weight distributions are shown in Table 8. Similar statistics for p388 dataset are shown in Table 9. Next, consider the comparative performance between our proposed approaches in the OVCAR-8 dataset. Next, consider the comparative performance between our proposed approaches in the OVCAR-8 dataset. For example, the number of candidates generated in UGMINE-GWU and UGMINE-RMU are 9139 and 880, respectively, for the minimum utility threshold of δ = 8% in OVCAR-8 dataset. From the bar charts in Fig. 8a and c, it is clear that UGMINE-RMU generates a significantly smaller number of candidates than UGMINE-GWU. This is because UGMINE-RMU has a tighter pruning condition. This explains why UGMINE-RMU performs better than UGMINE-GWU in both normal and log-normal distributions in terms of runtime performance.

Observed from Fig. 8b and d, the runtime performance gap between our proposed approaches is quite significant for lower thresholds as compared to higher thresholds, both in terms of runtime and search space reduction. This is because more false candidates are generated when the threshold is low. So, the UGMINE-RMU pruning technique becomes more effective than UGMINE-GWU for lower thresholds as this pruning technique efficiently prunes the search space by eliminating false candidates.

In Fig. 8, the performance gap is wider in the normal distribution than in the log-normal distribution, both in terms of runtime and search space reduction. As the shape of the log-normal distribution is right-skewed, generation of values lower than the mean tends to be more probable than generating values higher than the mean in this distribution. This explains why the assignment of both internal and

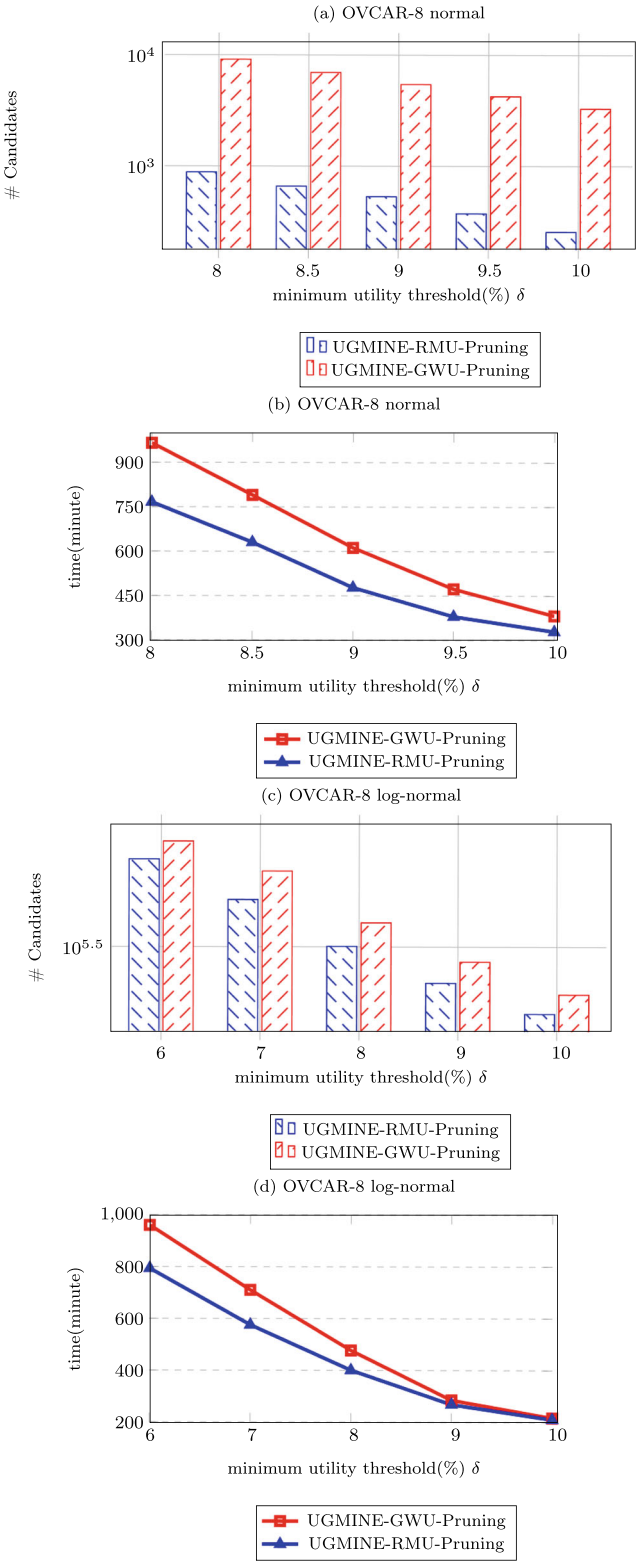


Fig. 8 Search Space Reduction and Runtime Analysis: OVCAR-8

[1] https://pubchem.ncbi.nlm.nih.gov/

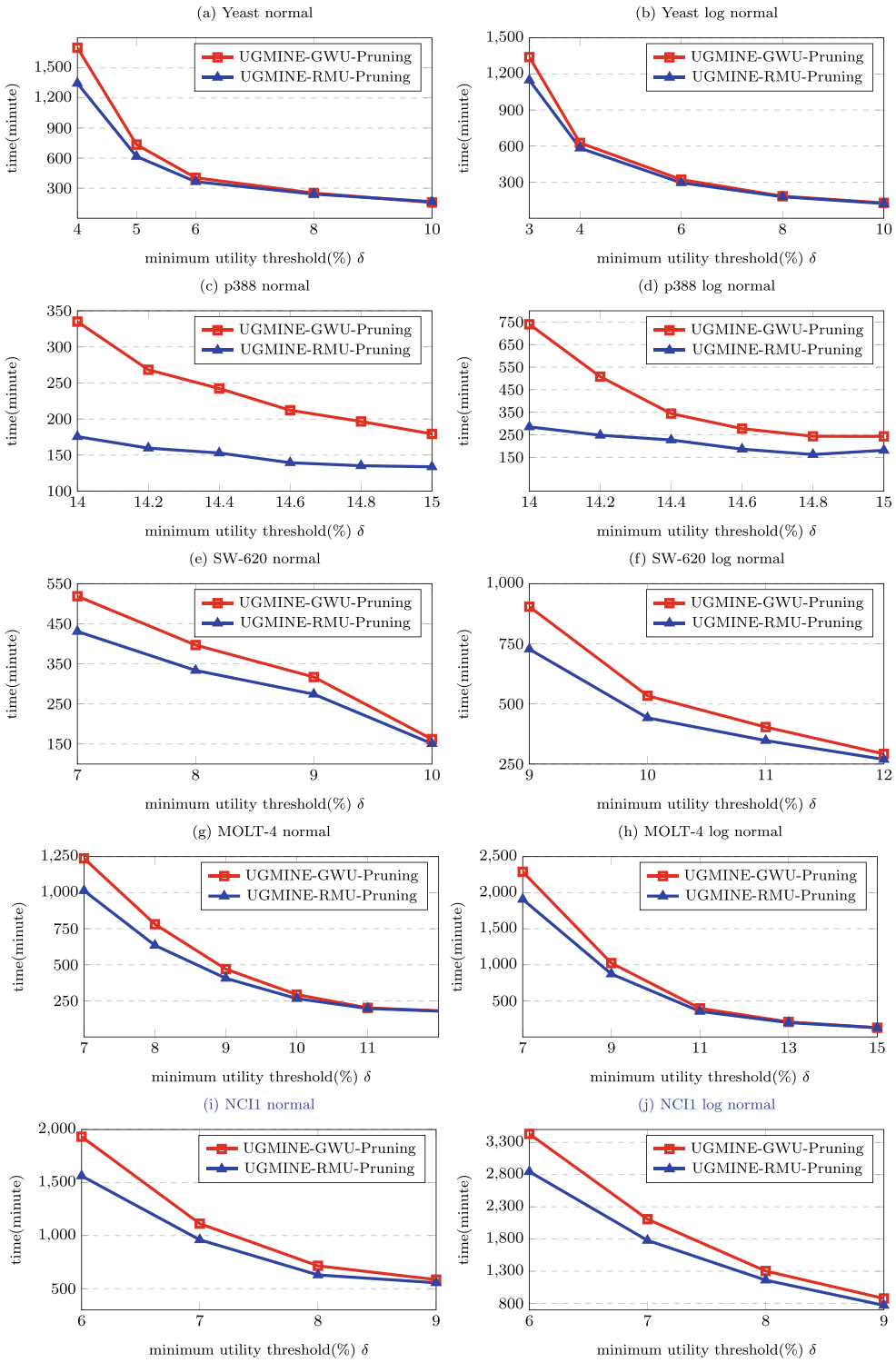


Fig. 9 Runtime Analysis: Yeast, p388, SW-620, MOLT-4, NCI1

external utility to edges takes smaller values in the log-normal distribution. Due to the presence of the highly-frequent smaller values in internal utility and external utility, the difference between the upper bounds of the UGMINE-RMU and UGMINE-GWU pruning techniques is smaller in the log-normal distribution than in the normal distribution. Hence, the difference between the numbers of candidates generated between the two approaches is relatively smaller in the log-normal distribution. This is a major reason behind the performance difference in two different distributions.

Next, we show similar comparative runtime performance in Yeast, p388, SW-620, MOLT-4 and NCI1 datasets in Fig. 9 with both the normal and log-normal distributions. In all these datasets, UGMINE-RMU-Pruning outperformed the UGMINE-GWU-Pruning in terms of runtime (Tables 10 and 11). As the threshold decreases, the performance difference between the two proposed approaches becomes clearer. With a lower threshold, more high utility subgraphs are generated. As a result, pruning the search space using the UGMINE-RMU-Pruning reduces the number of false candidates in the search space, which significantly improves the runtime in both the normal and log-normal distributions of internal and external utility.

However, the performance gap between UGMINE-RMU-Pruning and UGMINE-GWU-Pruning varies across different datasets. Observed from Fig. 9a-d, there is a significant performance gap between the two proposed approaches in the p388 dataset when compared to the yeast dataset. There can be two reasons for this behavior. First, the existence of fewer edges adjacent to the nodes not in the rightmost path can be a reason behind the lower performance improvement in the yeast dataset. Second, the number of graphs in the p388 and yeast datasets are 2298 and 9568, respectively. As the total number of graphs in the yeast dataset is very large, it takes a significant amount of runtime to calculate the GWU and RMU values for each candidate subgraph of the search space. Hence, the pruning methods make an insignificant difference in runtime performance. For these two reasons, the performance difference between the proposed approaches is insignificant in the yeast dataset. The performance gap in the SW-620, MOLT-4, and NCI1 datasets are average when compared to the yeast and p388 datasets. The aforementioned two reasons can also explain this average performance gap. The number of edges adjacent to the nodes that are absent from the rightmost path of the candidates is low. So, the RMU-pruning technique does not reduce the upper bound pruning value by a large margin. Again, the number of graphs in those datasets is moderate when compared to the Yeast dataset. Hence, the performance margins between the two proposed pruning techniques are moderate for the SW-620, MOLT-4 and

Table 10 Runtime and search space reduction statistics: SW-620

δ	UGMINE-GWU pruning			UGMINE-RMU pruning		
	Runtime (minute)	Candidates	HU Sub-Graphs	Runtime (minute)	Candidates	HU Sub-Graphs
(a) Normal Distribution						
7%	518.71	5417	65	430.80	1820	65
8%	396.89	4105	205	333.57	1620	205
9%	317.13	3248	254	274.35	1320	254
10%	162.21	1569	310	151.03	1120	310
(b) Log-normal Distribution						
9%	904.45	547381	136	727.63	417865	136
10%	534.05	369515	91	442.18	287463	91
11%	403.79	257307	65	347.78	211362	65
12%	292.28	193673	44	269.34	162129	44

Table 11 Runtime and search space reduction statistics: MOLT-4

δ	UGMINE-GWU pruning			UGMINE-RMU pruning		
	Runtime (minute)	Candidates	HU Sub-Graphs	Runtime (minute)	Candidates	HU Sub-Graphs
(a) Normal Distribution						
7%	1237.36	422	305	1013.78	310	305
8%	781.36	1186	190	634.92	190	190
9%	470.22	1799	128	406.62	180	128
10%	293.70	2819	79	265.95	170	79
(b) Log-normal Distribution						
7%	2289.27	1094209	295	1907.44	840807	295
9%	1023.97	484568	121	873.27	377462	121
11%	396.19	235112	55	354.29	195215	55
13%	208.33	136320	21	198.34	117786	21

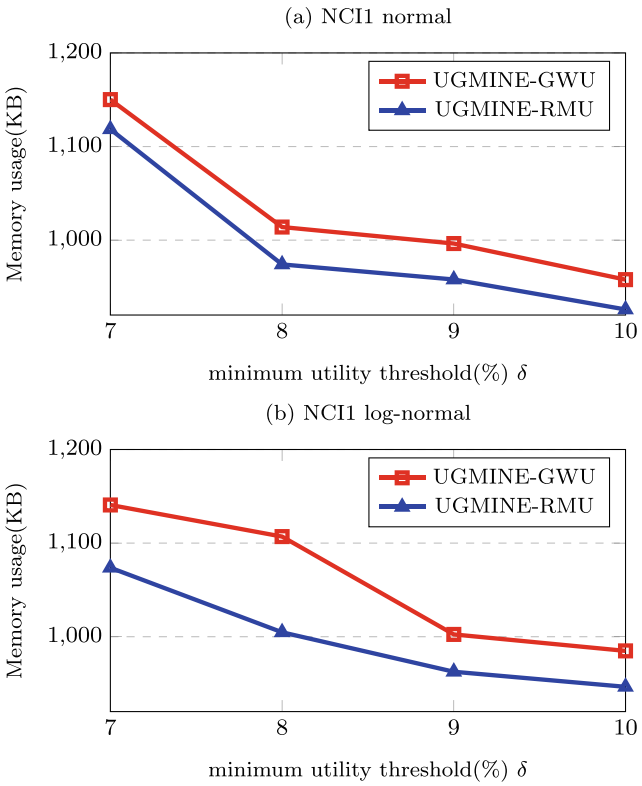


Fig. 10 Memory usage on NCI1

NCI1 datasets. Figure 10 shows memory usage of both the algorithms on the NCI1 dataset with normal and log-normal distribution. It is evident that the memory usage for UGMINE-RMU is lower than UGMINE-GWU. The low memory usage of UGMINE-RMU is explained by the low number of generated false candidates. We also observe that the memory usage is relatively higher for lower utility thresholds. The reason behind the difference is analogous to that of runtime. As the utility threshold decreases, the search space explodes, which leads to longer runtimes and higher memory costs. To summarize our experimental results, we have presented comparative performance analysis of our proposed algorithms using six different datasets. Experimental results and comparative performance analysis for both normal and log-normal distributions of internal and external utility show that the UGMINE-RMU pruning technique outperforms the baseline approach, UGMINE-GWU, in terms of the number of candidates generated in the search space, which leads to better performance in terms of runtime and memory usage. The improvement in performance is more notable when mining with a lower utility threshold, which is more challenging in utility-based pattern mining. Hence, UGMINE-RMU pruning is a better pruning technique than UGMINE-GWU pruning. Moreover, UGMINE-RMU scales well with a lower utility threshold.

5 Conclusions

In this work, we introduced a complete framework for utility-based graph mining. We also proposed an algorithm named UGMINE for extracting high utility subgraph patterns along with efficient pruning techniques. Experimental results show that our algorithm can efficiently mine high utility subgraphs. The generic framework and methods we proposed here are expected to be helpful for solving problems in areas such as analyzing web page access log networks, chemical structure databases, social networks, and anywhere graph databases are used. As ongoing and future work, we are exploring some tighter pruning techniques to support larger datasets. We are also modifying the framework based on the application such that summation, minimum, or other complex utility functions are used as a replacement of maximum. Moreover, we are examining negative utility or multi-edged negative utility. The source code of the algorithms is accessible at https://github.com/tfahim15/UGMINE.

Acknowledgements We would like to express our deep gratitude to the anonymous reviewers of this article. We believe their useful comments have played a significant role in improving the quality of this work, which was supported by Natural Sciences and Engineering Research Council of Canada (NSERC) and University of Manitoba.

References

1. Ahmed CF, Tanbeer SK, Jeong B, Lee Y (2009) Efficient tree structures for high utility pattern mining in incremental databases. IEEE Trans Knowl Data Eng 21(12):1708–1721
2. Ahmed CF, Tanbeer SK, Jeong BS (2010) A novel approach for mining high-utility sequential patterns in sequence databases. ETRI J 32(5):676–686
3. Ahmed CF, Tanbeer SK, Jeong BS, Lee YK (2011) HUC-Prune: an efficient candidate pruning technique to mine high utility patterns. Appl Intell 34(2):181–198
4. Bogdanov P, Mongiovì M, Singh AK (2011) Mining heavy subgraphs in time-evolving networks. In: 2011 IEEE 11th international conference on data mining. IEEE, pp 81–90
5. Cai C, Fu A, Cheng C, Kwong W (1998) Mining association rules with weighted items. In: IDEAS'98, pp 68–77
6. Chen Y, Zhao X, Lin X, Wang Y, Guo D (2019) Efficient mining of frequent patterns on uncertain graphs. IEEE Trans Knowl Data Eng 31(2):287–300
7. Chowdhury MES, Ahmed CF, Leung CK (2022) A new approach for mining correlated frequent subgraphs. ACM Trans Manag Inf Syst 13(1):9.1–9.28
8. Fournier-Viger P, Wu CW, Zida S, Tseng VS (2014) Fhm: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: International symposium on methodologies for intelligent systems. Springer, pp 83–92
9. Gan W, Lin JCW, Fournier-Viger P, Chao HC, Philip SY (2020) Huopm: High-utility occupancy pattern mining. IEEE Tran Cyber 50(3):1195–1208
10. Gan W, Lin JCW, Zhang J, Fournier-Viger P, Chao HC, Yu PS (2020) Fast utility mining on sequence data. IEEE Tran Cyber 51(2):487–500
11. Han J, Pei J, Yin Y, Mao R (2004) Mining frequent patterns without candidate generation: a frequent-pattern tree approach. In: Data mining and knowledge discovery, vol 8. Springer, pp 53–87
12. Islam MA, Ahmed CF, Leung CK, Hoi CS (2018) WFSM-MaxPWS: an efficient approach for mining weighted frequent subgraphs from edge-weighted graph databases. In: Pacific-Asia conference on knowledge discovery and data mining. Springer, pp 664–676
13. Jiang C, Coenen F, Zito M (2010) Frequent sub-graph mining on edge weighted graphs. In: International conference on data warehousing and knowledge discovery. Springer, pp 77–88
14. Khare A, Goyal V, Baride S, Prasad SK, McDermott M, Shah D (2017) Distributed algorithm for high-utility subgraph pattern mining over big data platforms. In: 2017 IEEE 24th international conference on high performance computing (HiPC). IEEE, pp 263–272
15. Kuramochi M, Karypis G (2001) Frequent subgraph discovery. In: Proceedings 2001 IEEE International Conference on Data Mining. IEEE, pp 313–320
16. Lan GC, Hong TP, Lee HY (2014) An efficient approach for finding weighted sequential patterns from sequence databases. Appl Intell 41(2):439–452
17. Lan GC, Hong TP, Lee HY, Wang SL, Tsai CW (2013) Enhancing the efficiency in mining weighted frequent itemsets. In: 2013 IEEE International conference on systems, man, and cybernetics, pp 1104–1108
18. Le NT, Vo B, Nguyen LB, Fujita H, Le B (2020) Mining weighted subgraphs in a single large graph. Inf Sci 514:149–165
19. Lin CW, Hong TP, Lu WH (2011) An effective tree structure for mining high utility itemsets. Expert Syst Appl 38(6):7419–7424
20. Lin JCW, Djenouri Y, Srivastava G, Li Y, Yu PS (2021) Scalable mining of high-utility sequential patterns with three-tier MapReduce model. ACM Trans Knowl Discov Data 16(3):60.1–60.26
21. Liu Y, Liao WK, Choudhary A (2005) A two-phase algorithm for fast discovery of high utility itemsets. In: PAKDD. Springer, pp 689–695
22. Malliaros FD, Skianis K (2015) Graph-based term weighting for text categorization. In: IEEE/ACM International conference on advances in social networks analysis and mining, pp 1473–1479
23. Nouioua M, Fournier-Viger P, Wu CW, Lin JCW, Gan W (2021) FHUQI-Miner: Fast high utility quantitative itemset mining. Appl Intell 51:6785–6809
24. Pei J, Han J, Mortazavi-Asl B, Pinto H, Chen Q, Dayal U, Hsu MC (2001) PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In: Proceedings 17th international conference on data engineering. IEEE, pp 215–224
25. Pramanik S, Goswami A (2021) Discovery of closed high utility itemsets using a fast nature-inspired ant colony algorithm. Appl Intell:1–17
26. Preti G, Lissandrini M, Mottin D, Velegrakis Y (2018) Beyond frequencies: Graph pattern mining in multi-weighted graphs. In: EDBT, pp 169–180
27. Rozenshtein P, Gionis A (2019) Mining temporal networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. ACM, pp 3225–3226
28. Singh K, Singh SS, Kumar A, Biswas B (2019) TKEH: an efficient algorithm for mining top-k high utility itemsets. Appl Intell 49(3):1078–1097
29. Singh K, Singh SS, Kumar A, Shakya HK, Biswas B (2018) CHN: an efficient algorithm for mining closed high utility itemsets with negative utility. IEEE Trans Knowl Data Eng:1–1

30. Song W, Zheng C, Huang C, Liu L (2021) Heuristically mining the top-k high-utility itemsets with cross-entropy optimization. Appl Intell:1–16
31. Srikant R, Agrawal R (1996) Mining sequential patterns: Generalizations and performance improvements. In: International conference on extending database technology. Springer, pp 1–17
32. Srikant R, Vu Q, Agrawal R (1997) Mining association rules with item constraints. In: KDD'97, pp 67–73
33. Tao F, Murtagh F, Farid M (2003) Weighted association rule mining using weighted support and significance framework. In: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, pp 661–666
34. Truong T, Duong H, Le B, Fournier-Viger P (2019) Efficient vertical mining of high average-utility itemsets based on novel upper-bounds. IEEE Trans Knowl Data Eng 31(2):301–314
35. Tung N, Nguyen LT, Nguyen TD, Vo B (2021) An efficient method for mining multi-level high utility itemsets. Appl Intell:1–22
36. Vo B, Coenen F, Le B (2013) A new method for mining frequent weighted itemsets based on WIT-trees. Expert Syst Appl 40(4):1256–1264
37. Wale N, Watson IA, Karypis G (2008) Comparison of descriptor spaces for chemical compound retrieval and classification. Knowl Inf Syst 14(3):347–375
38. Wang JZ, Chen YC, Shih WY, Yang L, Liu YS, Huang JL (2020) Mining high-utility temporal patterns on time interval–based data. ACM Trans Intell Syst Technol (TIST) 11(4):43:1–43:31
39. Wang JZ, Huang JL (2018) On incremental high utility sequential pattern mining. ACM Trans Intell Syst Technol (TIST) 9(5):55:1–55:26
40. Wang W, Yang J, Yu PS (2000) Efficient mining of weighted association rules (WAR). In: Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining, pp 270–274
41. Wu JMT, Lin JCW, Tamrakar A (2019) High-utility itemset mining with effective pruning strategies. ACM Trans Knowl Discov Data 13(6):58.1–58.22
42. Yan X (2002) Han, j.: gspan: graph-based substructure pattern mining. In: ICDM. IEEE, pp 721–724
43. Yang J, Su W, Li S, Dalkilic MM (2012) WIGM: discovery of subgraph patterns in a large weighted graph. In: Proceedings of the 2012 SIAM International Conference on Data Mining. SIAM, pp 1083–1094
44. Yin J, Zheng Z, Cao L (2012) USpan: an efficient algorithm for mining high utility sequential patterns. In: Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. ACM, pp 660–668
45. Yun U (2008) A new framework for detecting weighted sequential patterns in large sequence databases. Knowl-Based Syst 21(2):110–122
46. Yun U, Leggett JJ (2006) WSpan: Weighted sequential pattern mining in large sequence databases. In: 2006 3rd international IEEE conference intelligent systems, pp 512–517
47. Kim H, Yun U, Baek Y, Kim J, Vo B, Yoon E, Fujita H (2021) Efficient list based mining of high average utility patterns with maximum average pruning strategies. Inf Sci 543:85–105
48. Gan W, Lin JCW, Zhang J, Chao HC, Fujita H, Yu PS (2020) ProUM: Projection-based utility mining on sequence data. Inf Sci 513:222–240
49. Truong T, Duong H, Le B, Fournier-Viger P, Yun U, Fujita H (2021) Efficient algorithms for mining frequent high utility sequences with constraints. Inf Sci 568:239–264

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Md. Tanvir Alam received his B.Sc.(Hons.) in Computer Science and Engineering from the Department of Computer Science and Engineering, University of Dhaka in 2020. He is currently pursuing his M.S. degree in Computer Science and Engineering from the Department of Computer Science and Engineering, University of Dhaka. His research interests are graph mining, high utility pattern mining.



Amit Roy completed his B.Sc.(Hons.) degree in Computer Science and Engineering from University of Dhaka, Bangladesh in 2020. His current research interests include data mining and deep learning.



Chowdhury Farhan Ahmed received the B.Sc.(Hons.), M.Sc., degrees in Computer Science from the University of Dhaka, Bangladesh in 2000 and 2002 respectively, and the PhD degree in Computer Engineering from the Kyung Hee University, South Korea in 2010. He was a postdoctoral research fellow in the ICube Laboratory, University of Strasbourg, France from 2013 to 2015. He worked as a visiting scholar in the Faculty of Engineering and Information Technology, University of Technology Sydney, Australia in March, 2019. Since 2004, he has been working as a faculty member (and as a Professor since 2016) in the Department of Computer Science and Engineering, University of Dhaka, Bangladesh. His research interests are in the areas of data mining, knowledge discovery and machine learning.



Md. Ashraful Islam received his B.Sc.(Hons.) and M.S. degrees in Computer Science and Engineering from University of Dhaka, Bangladesh in 2015 and 2017 respectively. He is currently working as a faculty member in the Department of CSE, University of Dhaka, Bangladesh. His current research interests include data mining, big data analytics and deep learning.



Carson K. Leung received his B.Sc.(Hons.), M.Sc., and Ph.D. from the University of British Columbia, Vancouver, Canada. He is currently a Professor at University of Manitoba, Canada, at which he is also the founder and director of the Database and Data Mining Lab. Over the past few years, he has taken different roles in the Organizing Committees of many refereed international conferences. For instance, he has served as a General Chair for IEEE SmartData 2018, IEEE DSCI 2021, IEEE DSS 2021 and IEEE CBDCom 2022; a Program Chair for IEEE iThings 2015, IEEE BigDataSE 2016, IEEE HPCC 2016, IDEAS 2020 and IEEE CBDCom 2021; an organizing committee member of ACM SIGMOD 2008, IEEE ICDM 2011 & 2020, IEEE/ACM ASONAM 2014, IEEE/ACM DSAA 2016, 2020 & 2022, ACM CIKM 2018 and IEEE BigComp 2020-2022. He has also served as a PC member of numerous conferences including ACM KDD, ECML/PKDD and PAKDD. He is a senior member in IEEE and ACM, and an IEEE Computer Society Distinguished Contributor. He has contributed more than 300 refereed publications in the research areas of databases, data mining, social network mining, big data analytics, visual analytics, and data science.